RENT ESTIMATION SYSTEM

Full Name: Giray Doruk Yurtseven
University / Department: Halic University – Software Engineering
Project Title: Rent Estimation System
Project Code Repository: https://github.com/giraydorukyurt7/Rent-Estimation-System

Table of Contents:

1. Introduction
2. Dataset
3. Data Analysis (Feature Engineering and Visualization)
4. Algorithms and Ensemble Methods
5. Results Analysis
6. References

1. Introduction

The aim of this study is to estimate house rental prices in Turkey by using machine learning methods on a publicly available housing dataset. By modeling real-world data, the project evaluates both feature engineering strategies and comparative model performance across multiple regression algorithms.

In addition to single-model regression performance, the study also investigates whether ensemble-based prediction can improve estimation accuracy. For this reason, both individual regressors and ensemble combinations were systematically analyzed and compared.

2-Dataset

The dataset used in this project is the public "Turkish House Rent Prediction" dataset published on Kaggle [1]. It is provided in two separate parts: a training set with 6,166 observations and a test set with 2,642 observations, each containing 16 columns. Because the raw data required extensive preprocessing and exploratory analysis, the two parts were first merged and then split again after preprocessing.

The original dataset includes the following variables: City, Town, Neighborhood, Apartment Type, House Type, House Age, House Size, Room Count, Floor, Furniture, Bathroom Count, Hand, Heater Type, Heater Fuel, and Price. In addition, the unnecessary extra index column (`Unnamed: 0`) was removed.

3. Data Analysis (Feature Engineering and Visualization)
Before the analysis stage, the original training and test sets were merged, resulting in a combined matrix of size (8808, 16). Although several variables contained numerical information, many of them were stored in string-based or mixed formats. Therefore, a comprehensive preprocessing and feature engineering pipeline was applied.

3.1 Feature Engineering

- **Room Count:** Apartment layouts such as "3+1" and "2+1" were first analyzed by frequency. Rare room-count categories with fewer than 10 observations were removed. Then, the `Room Count` column was split into two separate numerical columns: `Rooms` and `Living Rooms`.

- **House Age:** The category representing brand-new houses was converted to 0. After conversion, house-age categories with fewer than 10 observations were removed in order to reduce noise from extremely rare values.

- **House Size:** Non-numeric entries were removed first. Then, values such as "120 m2" were converted into a numeric column named `House Size m2`. Based on outlier analysis, only houses with 45 <= `House Size m2` <= 250 were retained.

- **Price:** Based on distribution and outlier analysis, only listings with 6000 <= `Price` <= 40000 were retained.

- **Floor:** The `Floor` column was originally stored as a string and also included special categories such as middle floor, raised ground floor, and semi-basement. These values were mapped into a numerical column named `Floor Level` using rule-based conversion. Listings with labels indicating “and above” were capped at 21.

- **Location Variables:** Direct one-hot encoding of `Neighborhood`, `Town`, and `City` was not suitable because the same names may appear in different cities or towns. To address this, more specific combined location variables were created. `Neighborhood Grouped` was formed from “City - Town - Neighborhood”, and `Town Grouped` was formed from “City - Town”. Then, low-frequency categories were grouped as `Other`, using thresholds of 20 for neighborhoods, 20 for towns, and 10 for cities.

- **Apartment Type and House Type:** These two columns showed the same structure in a crosstab analysis, so `Apartment Type` was removed. Then, `House Type` was restricted to the following valid categories: `Apartment`, `Villa`, `Residence`, `Detached House`, and `Summer House`.

- **Heater Fuel:** Rare fuel types were removed. The analysis continued with listings that contained `Natural Gas`, `Electricity`, or missing values. Missing values were later filled as `Unknown`.

- **Heater Type:** Among 15 different categories, the most frequent and sufficiently represented types were retained: `Combi Boiler`, `Air Conditioning`, `Central Heating`, `Central Heating (Heat Metered)`, `Underfloor Heating`, and `No Heating`.

- **Hand / IsNew:** Rows labeled with the structurally inconsistent category were removed, and missing values in `Hand` were assigned as `Second Hand`. Then, the variable was converted into a binary feature named `IsNew`, where `Brand New` = 1 and `Second Hand` = 0.

- **Furniture:** Missing values in `Furniture` were filled with `False`.

- **Bathroom Count:** Missing values were filled with 1.0, and then listings with `Bathroom Count` > 4 were removed.

3.2 Correlation Analysis:

The correlation analysis of numerical features with `Price` showed that the strongest positive relationships were obtained for `Bathroom Count`, `House Size m2`, and `Rooms`. The correlation values are presented below.

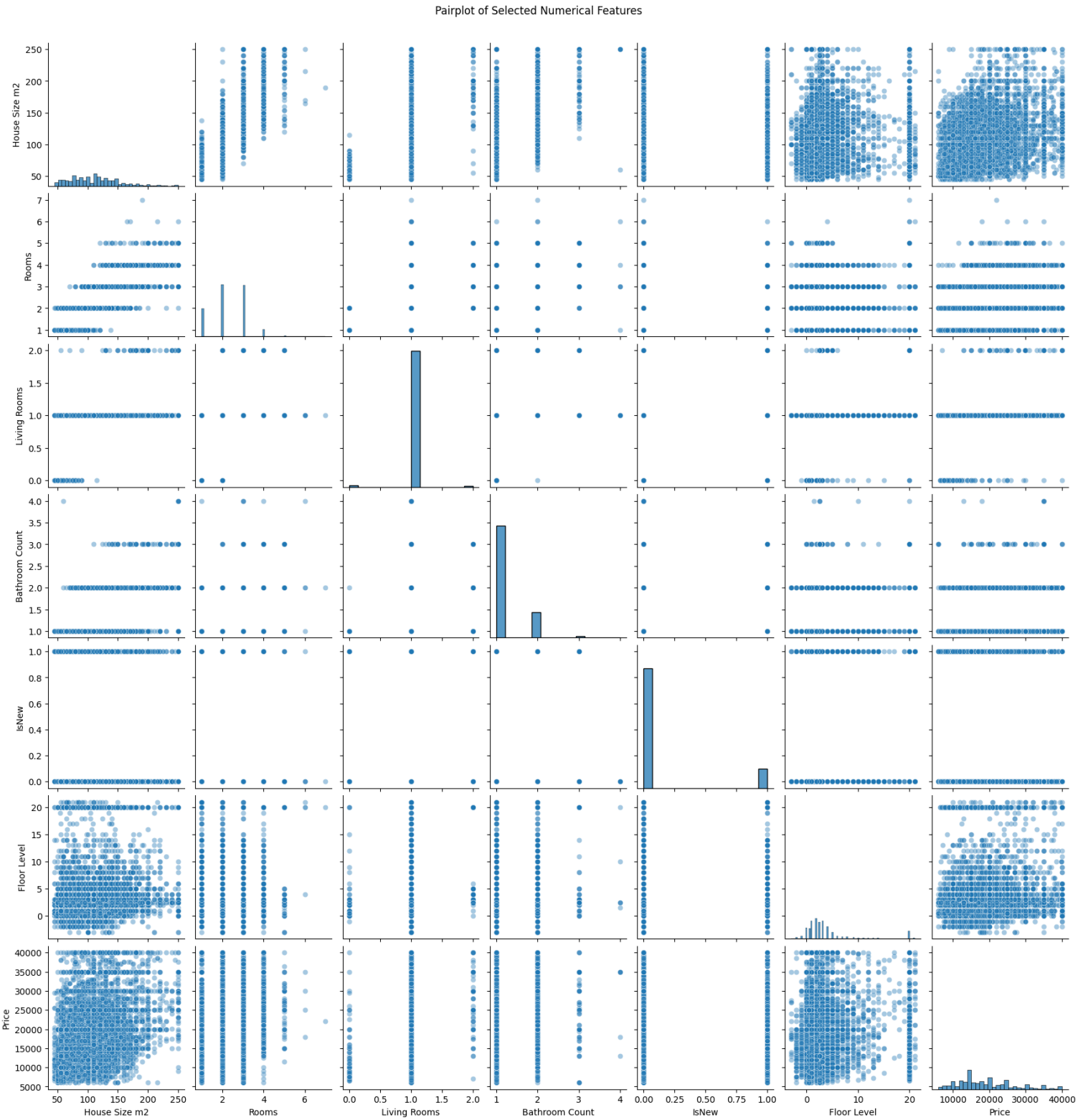
Correlation of Numerical Features with Price	Price
Price	1.000000
Bathroom Count	0.332318
House Size m2	0.316582
Rooms	0.283462
Living Rooms	0.121039
Floor Level	0.086650
House Age	0.039236
IsNew	0.015240

Among the numerical variables, `Bathroom Count` and `House Size m2` showed the highest correlation with `Price`, while `IsNew` had a relatively weak linear relationship. This suggests that property size and bathroom count contribute more strongly to rental price estimation than the new-versus-second-hand distinction.

3.3 Visualization of Distributions with a Pairplot

To better understand the distributions of selected numerical variables and their relationships with the target variable, a pairplot was generated using the features `House Size m2`, `Rooms`, `Living Rooms`, `Bathroom Count`, `IsNew`, `Floor Level`, and `Price`.

This visualization provides an intuitive overview of both marginal distributions and pairwise relationships, making it easier to detect clustering patterns, skewness, and potential nonlinear trends among the most relevant numerical variables.



3.4 Final Data Preprocessing

In order to make the data suitable for machine learning models, one-hot encoding was applied to the categorical columns `House Type`, `Heater Type`, `Heater Fuel`, `Neighborhood Grouped`, `Town Grouped`, and `City Grouped`. After all preprocessing and feature engineering steps, the final data matrix reached a shape of (6624, 178).

Then, the data was split again by using the `train_test_split` function with an 80%–20% ratio. As a result, the training set contained 5,299 observations and the test set contained 1,325 observations.

4. Algorithms and Ensemble Methods

4.1 Algorithms

In this study, seven regression algorithms with different characteristics were evaluated: KNN, SVR, DecisionTree, RandomForest, GBM, XGBoost, and LightGBM. For each algorithm, hyperparameter tuning was carried out by using GridSearchCV. After training, all models were evaluated by using RMSE and MAPE as the main performance metrics.

These models were selected in order to compare different learning behaviors, including distance-based, kernel-based, tree-based, and boosting-based regression strategies.

4.2 Ensemble Methods

To go beyond single-model performance, two different ensemble approaches were applied: Averaging and Stacking.

1. Averaging

In this method, the predictions of multiple trained models are combined by taking their arithmetic mean. This is a simple yet effective ensemble strategy, especially when the participating models make different types of errors.

2. Stacking

In the stacking approach, the predictions of base regressors are combined through a meta-model. In this project, the ensemble was implemented by using `StackingRegressor` with `LinearRegression` as the final estimator. This method aims to learn how to optimally combine the outputs of different base models in order to improve final prediction accuracy.

4.3 Combinations and Number of Experiments

Instead of evaluating only a fixed ensemble structure, all possible model combinations containing 3 or more models were tested. With 7 base models and 2 ensemble types, the total number of ensemble configurations is:

$2 * [C(7,3) + C(7,4) + C(7,5) + C(7,6) + C(7,7)] = 198$ ensemble models.

This exhaustive comparison made it possible to analyze not only whether ensemble learning improved performance, but also which model combinations were the most effective for the rent estimation task.

5. Results Analysis

5.1 Single Models

Model	RMSE	MAPE	Best Params	Training Time (s)
XGBoost	4744.075053	0.203971	{'colsample_bytree': 0.8, 'gamma': 0, 'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 200, 'reg_alpha': 0.1, 'subsample': 0.8}	560.906501
GBM	4827.084521	0.209387	{'learning_rate': 0.1, 'loss': 'squared_error', 'max_depth': 7, 'max_features': 'sqrt', 'n_estimators': 200, 'subsample': 0.8}	191.147500
RandomForest	4786.827367	0.210086	{'bootstrap': False, 'criterion': 'squared_error', 'max_depth': None, 'max_features': 'sqrt', 'min_samples_split': 5, 'n_estimators': 200}	322.959500
LightGBM	4953.731574	0.218108	{'feature_fraction': 0.8, 'learning_rate': 0.2, 'max_depth': 4, 'n_estimators': 200, 'reg_alpha': 0.1, 'reg_lambda': 0, 'subsample': 0.8}	61.733998
SVR	5398.246758	0.228871	{'C': 100, 'epsilon': 0.1, 'gamma': 'scale', 'kernel': 'linear'}	287.153000
DecisionTree	5899.696648	0.260703	{'criterion': 'squared_error', 'max_depth': 10, 'max_features': None, 'min_samples_leaf': 4, 'min_samples_split': 10}	16.535501
KNN	6419.426173	0.292046	{'n_neighbors': 11, 'p': 1, 'weights': 'distance'}	35.341001

Among the single-model regressors, XGBoost achieved the best performance with a MAPE of 0.203971 and an RMSE of 4744.075053.

This result indicates that gradient-boosting-based tree ensembles were more effective than the other standalone regression methods tested in this study. In contrast, simpler instance-based and shallow tree-based approaches such as KNN and DecisionTree produced weaker results, especially in terms of error magnitude.

5.2 Ensemble Models

The top 10 ensemble models and the top-ranked averaging ensemble model, starting from the best, are reported in the table below.

Ensemble Type	RMSE	MAPE	Models used
Stacking	4657.24	0.1994	DecisionTree + RandomForest + XGBoost
Stacking	4661.00	0.1999	RandomForest + GBM + XGBoost
Stacking	4665.10	0.1999	KNN + DecisionTree + RandomForest + XGBoost
Stacking	4663.06	0.2000	KNN + RandomForest + XGBoost
Stacking	4667.92	0.2005	KNN + DecisionTree + SVR + RandomForest + XGBoost
Stacking	4660.99	0.2005	KNN + SVR + RandomForest + GBM + XGBoost
Stacking	4653.16	0.2005	KNN + RandomForest + GBM + XGBoost
Stacking	4673.64	0.2006	DecisionTree + SVR + RandomForest + XGBoost
Stacking	4667.95	0.2007	SVR + RandomForest + GBM + XGBoost
Stacking	4669.41	0.2007	KNN + DecisionTree + SVR + RandomForest + GBM + XGBoost
Averaging	4660.31	0.2031	RandomForest + GBM + XGBoost

Among the ensemble models, the best overall result was obtained by the Stacking ensemble composed of `DecisionTree + RandomForest + XGBoost`, which achieved an RMSE of 4657.24 and a MAPE of 0.1994.

The best Averaging ensemble in the ranking was `RandomForest + GBM + XGBoost`, with an RMSE of 4660.31 and a MAPE of 0.2031.

Overall, both ensemble methods reduced prediction error compared with several single-model baselines. In particular, the top-ranking Stacking configurations outperformed the Averaging configurations, indicating that learning a meta-level combination of model outputs provided a stronger predictive structure for this problem.

These findings show that combining complementary regressors can improve rental price estimation performance beyond individual models.

6. References

[1] Atakan, A. (2024). Turkish House Rent Prediction Dataset [Dataset]. Kaggle. https://www.kaggle.com/datasets/atakanak/turkish-house-rent-prediction-dataset/data